THE UNIVERSITY OF THE SOUTH PACIFIC

CS241: Software Design & Implementation
SCIMS, FSTE

Final Examination
Semester II, 2020

F2F

Duration of Exam: 3 hours + 10 minutes

Reading Time: 10 minutes

Writing Time: 3 hours

Total Marks: 45

The exam covers the following learning outcomes partially:
CLO – 1: Develop software design document (~70%)
CLO – 2: Develop a simple software system based on the design document (~30%)

Instructions:

1. Write your answers in the space provided in this Question Paper.
2. Answer all questions. There are 7 questions and all questions are compulsory.
3. This exam is worth 40% of your overall mark. The minimum mark to pass the final exam is 16/40.
4. The total number of pages including this cover sheet is 14.
5. This is a closed book exam. No printed materials and electronic devices are allowed.

ID:_____ **Name:**_____

SeatNo: _____**Campus:** _____

This page is intentionally left blank

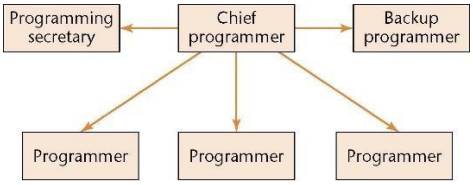
ID: _____

1. The students of CS241 were given the assignment for making a web application that uses MVC architecture. One group of students distributed the task evenly within the group and managed the software development process well. During the demo individual programs with stubs were working fine but the overall project was not in a working condition. What went wrong? What kind of integration and system testing would you choose and why? (2+2 marks)

The issue likely stems from the **lack of integration between the individual components** of the application. While each student may have worked on their own part (Model, View, or Controller) independently, they didn't ensure that these parts would work together in the final application. This is a common problem when using the **stub approach**, where parts of the system are initially replaced with placeholders or stubs that mimic real functionality but aren't fully implemented. As a result, the overall system likely didn't function correctly when it was time to integrate everything because the individual components weren't properly aligned or tested for interaction with each other.

Integration and System Testing:

1. **Integration Testing**:
   - **Purpose**: To ensure that the different components of the system (Model, View, Controller) work together as expected.
   - **Approach**: Focus on testing the interaction between the components. For instance, check that the **Controller** correctly updates the **Model**, and that the **Model** communicates the changes to the **View**. This would involve testing the data flow and user interactions across components.
   - **Why**: Since the individual parts might have worked in isolation but not when integrated, integration testing is necessary to identify and resolve issues in how the components interact.
2. **System Testing**:
   - **Purpose**: To validate that the entire application works as expected in a real-world environment.
   - **Approach**: Perform end-to-end testing of the application, including all the features. This could involve running test cases that simulate actual usage (e.g., submitting forms, displaying data) to ensure the system meets all functional and non-functional requirements.
   - **Why**: System testing ensures that the complete web application works as a whole, including proper error handling, user interactions, and performance under real-world conditions. This helps identify issues not just with integration, but with the overall functionality and user experience.



Identify at least three impracticalities of the classical chief programmer team shown in the figure below. Note that the chief programmer is also the manager. (3 marks)

The chief programmer must be a highly skilled programmer and a successful manager
There is a shortage of highly skilled programmers
There is a shortage of successful managers
The qualities needed to be a highly skilled programmer are unlikely to be found in a successful manager, and vice versa
The back-up programmer must be as good as the chief programmer
The programming secretary does nothing but paperwork all day

3. What are thin and fat client models? What is the main disadvantage of the fat client model? Identify the correct model for MVC architecture? (1+1+1=3 marks)

Thin Client Model:

- **Definition**: In the thin client model, most of the processing is done on the server side, and the client (typically a lightweight application or web browser) primarily acts as a user interface. The client only handles user interactions and displays data, while the server processes the logic and manages the database.
- **Example**: A web application that runs in a browser, where the server handles all the computation and data management.

Fat Client Model:

- **Definition**: In the fat client model, the client handles most of the processing and logic, and the server mainly stores data and responds to requests. The client application is more complex and self-sufficient, with a significant portion of the business logic and user interface processing occurring on the client side.
- **Example**: A desktop application where the software runs locally, with only data synchronization happening between the client and the server.

Main Disadvantage of the Fat Client Model:

- **Disadvantage**: **Maintenance and Upgrades**. In a fat client model, since much of the logic is stored on the client side, upgrading or fixing bugs in the application requires updating the client software on each individual machine. This can be cumbersome and time-consuming, especially when dealing with large numbers of clients. Additionally, the clients may have varying configurations, which can lead to compatibility issues.

Correct Model for MVC Architecture:

- **Correct Model**: The **Thin Client Model** is generally more suitable for MVC architecture, especially in web applications. In MVC, the Model handles the business logic, the View presents the user interface, and the Controller handles user input and interactions. In a thin client setup, the server typically manages the Model and the Controller, while the client just handles the View (displaying the UI). This aligns well with the separation of concerns in MVC and makes the system easier to maintain and scale.

4. Why is OOP recommended for large information systems when OOP languages are slower than traditional structured programming languages like C. (2 marks)

Object-Oriented Programming (OOP) is recommended for large information systems because it provides a modular structure that promotes **reusability**, **scalability**, and **maintainability**. Although OOP languages (like Java or Python) may be slower than traditional languages like C, the benefits of organizing the system into objects and classes allow for easier management of complex systems. This approach helps in better handling of large codebases, simplifying updates and modifications, and reducing the risk of errors. Furthermore, **encapsulation**, **inheritance**, and **polymorphism** are key OOP principles that enable teams to work on different components independently, making it easier to handle large, evolving projects.

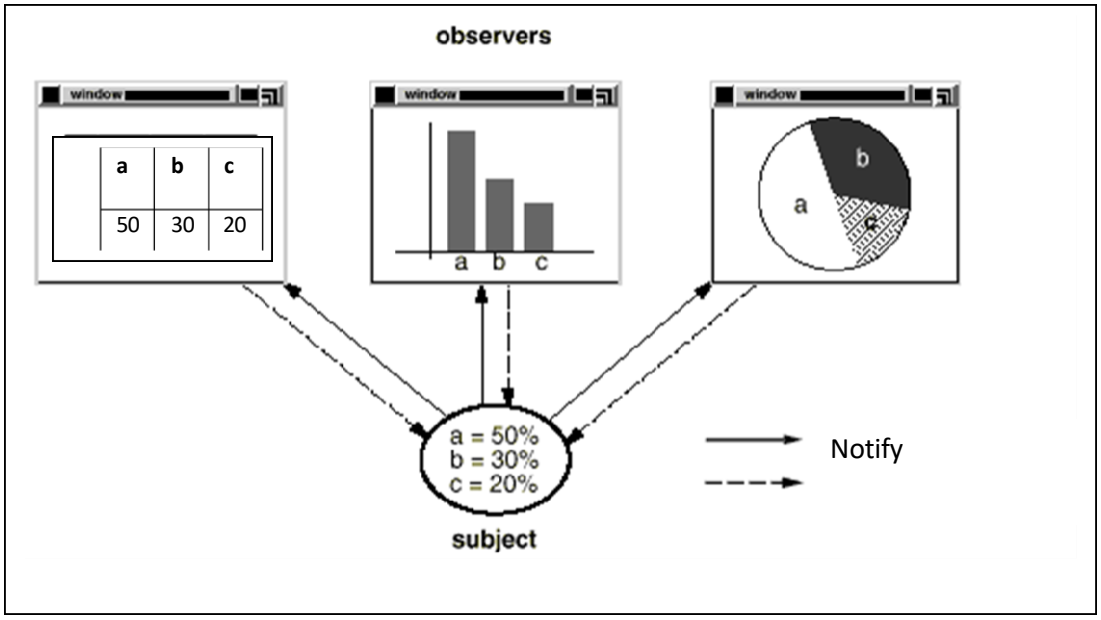
5. Explain two situations in which you would recommend against the application of an agile method for developing a software system? (3 marks)

1. **Well-Defined and Fixed Requirements**: If the software system has very **clear, stable, and well-defined requirements**, where the scope is unlikely to change, an **agile methodology** might not be the best choice. In such cases, traditional methods like the **Waterfall model** may be more effective because they provide a structured approach with a detailed upfront planning phase. Agile's iterative nature might lead to unnecessary adjustments and may not be efficient for projects where the requirements are unlikely to evolve.
2. **Large, Complex Systems with High Regulatory Constraints**: For systems that need to comply with strict **regulatory requirements** (e.g., healthcare, aerospace), or those that require **heavy documentation** for auditing and compliance, agile can become challenging. The constant iterations and lack of exhaustive documentation in agile can hinder compliance efforts. In such cases, a more **traditional, structured** methodology that emphasizes thorough documentation, process control, and predictable deliverables would be better suited.

6. The purpose of a design pattern is to define a particular behavior or technique that can be used as a building block for the construction of software where there is need to solve universal problems that developers commonly face. Observer Design Pattern is a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically. The generic class structure of observer interface and subject class is given below: (*total 10 marks, allocate approx. 30 minutes*)

```
interface IObserver{
        public void update();
}

class CSubject{
        private ArrayList< IObserver > observers =
                new ArrayList< IObserver >();

        public void addObserver(IObserver obs ){
                observers.add(obs);
        }

        public void removeObserver(IObserver obs){
                observers.remove(obs);
        }

        protected void notifyObservers() {
                for (int i=0; i < observers.size(); i++)
                        observers.get(i).update();
        }
}
```

Observer Pattern is very helpful in GUI design where a different graphical representation of the same data is required. The figure below shows different representations of student pass rates in the course CS241 where *a* indicates percentage of *A* graders, *b* indicates the percentage of *B* graders and *c* indicates the percentage of *C* graders.



As soon as the pass rate percentages are changed it should be reflected in all the graphs. This behavior implies that the table view, bar graph and pie chart are dependent on the data object and therefore should be notified of any change in its state. As the figure indicates all graphs (observers) are *observing* the pass rate data (subject).

Use the class definitions of the *Course*, *TableView*, *BargraphView* and *PiechartView* to answer the given questions.

Note: Dummy code are written in draw() methods to draw the graphs.

```
class Course {
	public String ID;
	public String title;

	private float Agraders;
	private float Bgraders;
	private float Cgraders;

	public void set_passrate(float a, float b, float c){
		Agraders = a;
		Bgraders = b;
		Bgraders = c;
	}
}
```

```
abstract class view{
	protected Course c;
	public void set_course (Course c){
		this.c = c;
	}
	public abstract void draw();
}

class TableView extends view{
	public void draw(){
		//code to draw this.c in table view
		System.out.println("Table view:\n");
		System.out.println(c);
	}
}

class BargraphView extends view {
	public void draw(){
		//code to draw this.c in bar graph
		System.out.println("Bar graph view:\n");
		System.out.println(c);
	}
}

class PiechartView extends view {
	public void draw(){
		//code to draw this.c in pie chart
		System.out.println("Pie chart view:\n");
		System.out.println(c);
	}
}
```

a) Briefly describe how design patterns help in software development. (2 marks)

Design patterns help in software development by providing proven, reusable solutions to common problems that developers face. They promote code modularity, maintainability, and reusability. By applying design patterns, developers can avoid reinventing the wheel and can create software that is easier to understand, maintain, and extend. They also facilitate communication among developers by using a shared vocabulary of design solutions.

Update the implementation of *Course* class to build a *subject* of observer pattern so that it can be observed by other class(es). Do not rewrite the entire code, just write the code inside the square brackets [ ]. (2 marks)

```
class Course [                                                    ]{
      …

      public void set_passrate(float a, float b, float c){
            Agraders = a;
            Bgraders = b;
            Bgraders = c;
            [

            notifyObservers();

            ]
      }
}
```

c) Build all the viewers, *observer* of the *Course* class so that as soon as the pass rate changes, the viewer objects are notified to draw the updated picture. Again write the code inside the square brackets [ ]. (2 marks)

```
abstract class view [                                    ]{
      …
      public void set_course (Course c){
            this.c = c;
            [


            ]
      }
      …
      public void update(){
            [
              draw();
            ]
      }
}
```

d) Develop the driver class to create a course object for CS241 where pass rates in the mid-semester test is 10% *A* grades, 8% *B* grades and 30% *C* grades. Draw this information in table view, bar graph view and pie chart view using appropriate methods of view classes. (2 marks)

```
public class Driver {
  public static void main(String[] args) {
    Course c = new Course();
    TableView tableView = new TableView();
    BargraphView bargraphView = new BargraphView();
    PiechartView piechartView = new PiechartView();

    c.addObserver(tableView);
    c.addObserver(bargraphView);
    c.addObserver(piechartView);

    c.set_passrate(10, 8, 30); // Set pass rates

    // Draw the updated views
    tableView.draw();
    bargraphView.draw();
    piechartView.draw();
  }
}
```

e) If the pass rates for this same course are changed to 10% *A* graders, 12% *B* graders and 40% *C* graders after a quick recheck of few papers, how can these pass rates be reflected in all the current views. Explain using Java code. (1 + 1 = 2 marks)

```
c.set_passrate(10, 12, 40);
```

7. Read the following case study to answer the given questions. (20 marks)

Case Study: USP Library System (allocate approx. 1 hour)

The web-based USP library system keeps track of library books. Information is maintained about both book titles and the individual book copies. Book titles maintain information about the title, author, publisher, and catalog number. Individual copies maintain copy number, edition, publication year, ISBN, book status (whether it is on the shelf or loaned out), and date due back in.

Currently, university students and staff are the patrons of the library, however, each has different privileges. The library keeps track of their records. To avoid redundancy of patrons' information, existing data from the enrollment system (for students) and the human resources system (for staff) are used. Basic information such as id, name, address, telephone number program enrolled, major/minor, job title and current status are stored.

Patrons have access to the library information to search for book titles and to see whether a book is available. They can also reserve a title by login to the system if all copies are checked out. When patrons bring books to the circulation desk, a clerk checks out the books on a loan. Clerks also check the books in.

Assume all the books have to be returned in the check-in process. The system must keep track of loan status as open, close or fine pending. Patrons are liable to pay fine for any late return of books. All books are loaned for 2 weeks for undergrad students and 4 weeks for staff/postgraduate students. Otherwise, they are liable to pay a fine of $1.00/day for any late return of books.

For the above case study answer the following questions:

1) Develop DFD – level 0 and level 1 diagram (4 marks)
2) What would be your preferred architecture if you build this system from scratch? Justify. (1 mark)

I would prefer a **layered architecture** (also known as a multi-tier architecture), with the following layers:

a. **Presentation Layer (UI Layer)**: Web interface for patrons, clerks, and administrators.

b. **Business Logic Layer (Service Layer)**: Handles all the business logic such as checking out books, calculating fines, and enforcing the rules of loaning books.

c. **Data Access Layer (DAL)**: Manages interactions with the database where book information, patron data, loan records, and fines are stored.

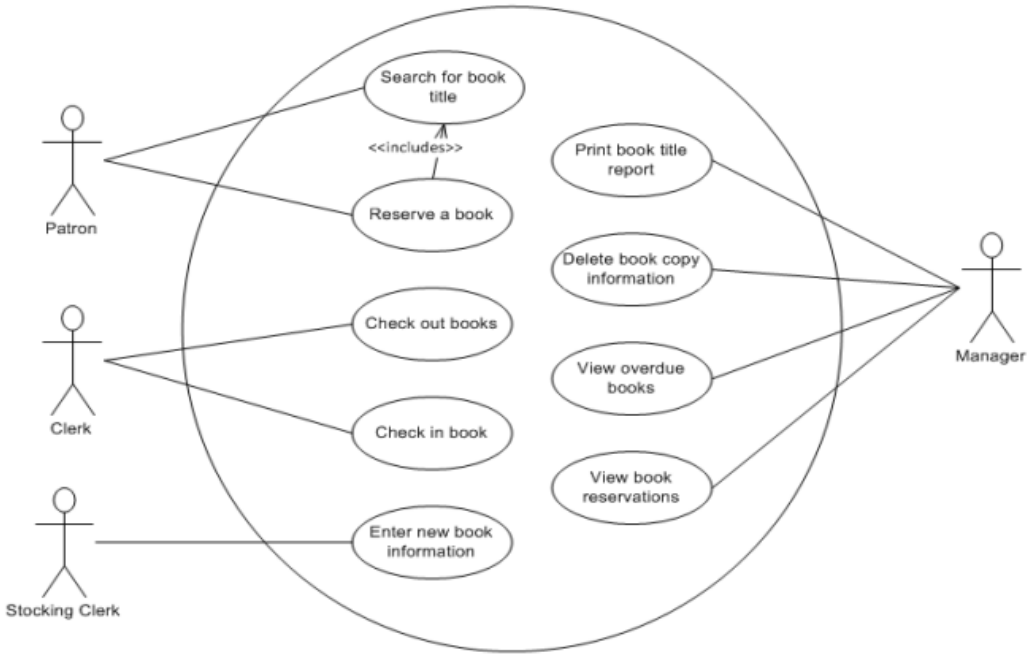
d. **Database Layer**: Where all the data is stored (book inventory, patron information, loan status, etc.).

Justification: This architecture promotes separation of concerns, where each layer is responsible for a specific functionality. It also provides maintainability and scalability, allowing the system to grow or change without affecting the whole system. The UI layer can be replaced with a different interface (e.g., mobile app) without changing the business logic.

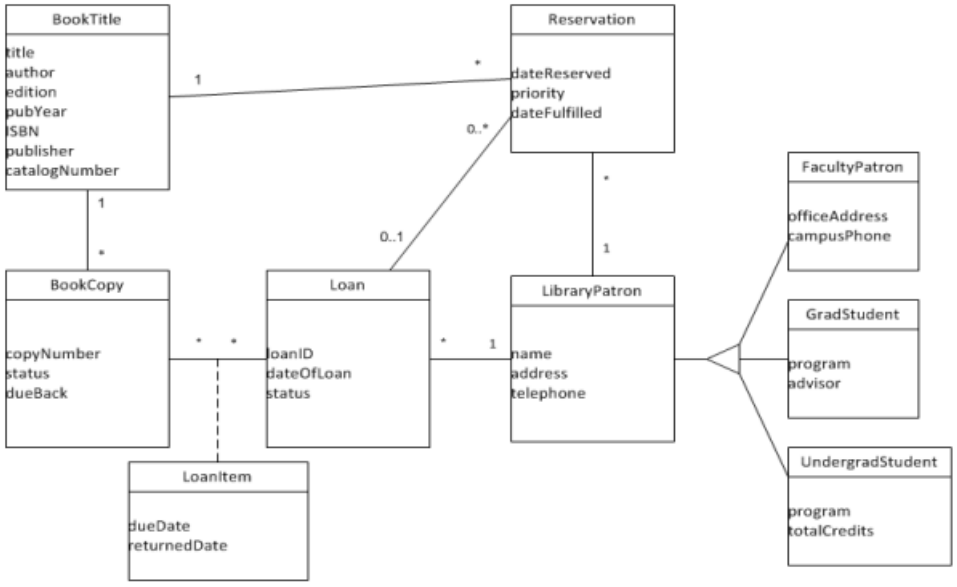
3) What kind of integration testing would you choose to develop this system? (1 mark)

I would choose **end-to-end integration testing** (also known as system integration testing). This involves testing the interactions between the various modules (book search, reservation, checkout, check-in, fines calculation, and payment) and ensuring that they work together as expected. Since the system involves multiple processes that depend on each other (e.g., checking out a book, calculating fines, and updating patron data), it's important to verify the whole system's functionality from start to finish

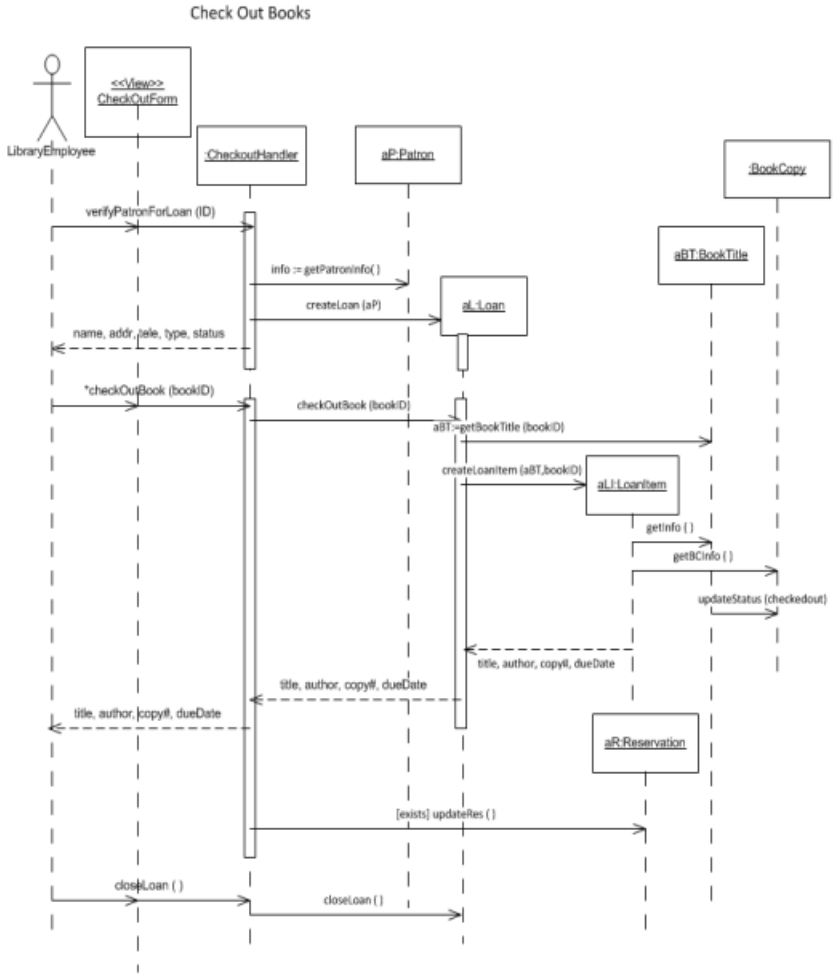
4) Model Use case diagram of the system (4 marks)



5) Model Analysis class diagram of the system. (5 marks)



6) Model the Sequence Diagram for books check-in scenario where the student has to pay the fine and he is paying the full amount. (5 marks)



THE END

Extra Sheet